

Exercices corrigés

AI Technologies, cours 1 à 3 et lab 1

BMEVIAUMSMA001, automne 2026. Rédigé le 2026-09-28.

À retenir

- **Énoncés** : 34 questions en 4 niveaux (cours 1, Python avancé, NumPy et Pandas, frameworks et labs), de la question de cours à la lecture de code.
- **Corrigés** : dans une partie séparée, avec la méthode et la source.

Statut. Ce ne sont pas des sujets officiels : aucun exemple de ZH n’a été distribué et son format est **À VÉRIFIER**. Les questions suivent les supports C1, C2, C3 et l’énoncé du lab 1 (L1) ; les corrigés renvoient aux numéros de concept de « AI Technologies, cours et méthodes ».

Sorties de code. Toutes obtenues en exécutant le code (Python 3.14, NumPy 2.5). PyTorch n’étant pas installé sur la machine de rédaction, les questions PyTorch se corrigent par le raisonnement et le calcul, sans sortie imprimée.

Table des matières

1	Énoncés	2
1.1	Niveau 1. Fondations et Python (cours 1)	2
1.2	Niveau 2. Python avancé (cours 2)	3
1.3	Niveau 3. NumPy et Pandas (cours 2)	5
1.4	Niveau 4. Frameworks et labs (cours 3, lab 1)	6
2	Corrigés	8
2.1	Niveau 1	8
2.2	Niveau 2	9
2.3	Niveau 3	11
2.4	Niveau 4	12

1. Énoncés

Niveau 1. Fondations et Python (cours 1)

Q1. Comparer l'IA symbolique et l'IA connexionniste selon quatre critères : époque, représentation de la connaissance, mécanisme de raisonnement, matériel. Pourquoi l'approche symbolique a-t-elle échoué sur les tâches de perception ?

Q2. Pour chaque tâche, dire s'il s'agit d'apprentissage supervisé ou non supervisé et écrire la quantité modélisée ($P(Y | X)$ ou $P(X)$) : (a) classer des radiographies étiquetées « sain » ou « malade » ; (b) repérer des paquets réseau anormaux sans aucun exemple d'attaque ; (c) générer de nouvelles images réalistes ; (d) prédire un score de risque à partir d'analyses sanguines ; (e) regrouper des clients selon leurs achats.

Q3. Décrire ce qui se passe entre `python main.py` et l'exécution de la première instruction. Donner ensuite les cinq opérations de la PVM pour `a + b` et expliquer pourquoi cette addition est plus lente qu'en C++.

Q4. Donner la sortie de ce programme.

```
a = [1, 2, 3]
b = a
b.append(4)
c = a[:]
c.append(5)
a = [0]
print(a, b, c)
```

Q5. Donner la sortie de :

```
print(True + 1, type({}), type(set()), (1) == (1,), 0 < 5 < 3)
```

Q6. Donner la complexité de : `l.append(x)`, `l.insert(0, x)`, `l.pop()`, `l.pop(0)`, `x in l`, `x in s` (`s` ensemble), `d[k]`. Expliquer le résultat attendu du devoir 3 du cours 1 (tester `-1 in data` sur une liste et sur un ensemble de 0 à 10 000 000).

Q7. Donner, dans l'ordre, les commandes qui créent un environnement `ai_lab_1`, l'activent sous Linux, installent `numpy` et `torch` et figent les versions. Qu'est-ce que l'activation modifie ? À quoi sert le fichier produit ?

Q8. La fonction `generate_spiral_data_python(n_points)` du cours 1 est appelée avec `n_points = 100`. Combien de points produit-elle, et combien d'appels à `math.sin`? Donner les quatre raisons pour lesquelles cette implémentation ne passe pas à l'échelle.

Niveau 2. Python avancé (cours 2)

Q9. Donner la sortie et l'expliquer.

```
class A:
    def __init__(self):
        self.__x = 1

class B(A):
    def __init__(self):
        super().__init__()
        self.__x = 2

b = B()
print(sorted(vars(b)))
```

Q10. Compléter : `__new__` reçoit ..., doit renvoyer ..., est appelé ...; `__init__` reçoit ..., doit renvoyer Donner les deux situations où redéfinir `__new__` est nécessaire.

Q11. Donner la MRO de `Child` et la sortie de `Child()`.

```
class Base:
    def __init__(self):
        print("Base")

class Left(Base):
    def __init__(self):
        print("Left")
        super().__init__()

class Right(Base):
    def __init__(self):
        print("Right")
        super().__init__()

class Child(Left, Right):
    def __init__(self):
        print("Child")
        super().__init__()

Child()
```

Q12. Dans l'exemple `SmartPhone(Camera, Phone)` du cours 2, `Camera` et `Phone` ont des constructeurs `(name, resolution)` et `(name, number)` qui appellent chacun `super().__init__(name)`. Que se passe-t-il à la création d'un `SmartPhone`? Quelle est la correction proposée par le cours, et quelle règle doit respecter chaque classe?

Q13. Donner la sortie de la dernière ligne, puis dire ce qui se passe si l'on retire la ligne `nonlocal count`.

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

c1 = make_counter()
c2 = make_counter()
print(c1(), c1(), c2())
```

Q14. Donner la sortie complète de ce programme, dans l'ordre.

```
def deco(f):
    print("decorating", f.__name__)
    def wrapper(*args):
        print("before")
        r = f(*args)
        print("after")
        return r
    return wrapper

@deco
def double(x):
    return 2 * x

print(double(5))
print(double.__name__)
```

Quelle ligne faut-il ajouter pour que la dernière instruction affiche `double`?

Q15. Avec le registre du cours 2 (`register_model` appliqué à `ResNet50` et `VGG16`), que contient `MODEL_REGISTRY`? Que produit

`create_model("BERT")`? Quel avantage ce motif a-t-il sur une fabrique écrite en `if` et `elif`?

Q16. Associer chaque situation à la socket Jupyter concernée : (a) un calcul long doit être interrompu ; (b) un `print` s'affiche sous la cellule ; (c) le code appelle `input()` ; (d) le frontend vérifie que le noyau répond ; (e) l'utilisateur exécute une cellule.

Q17. Un étudiant écrit ses résultats dans `/content/results.csv` sur Colab, ferme l'onglet et revient le lendemain : le fichier a disparu. Pourquoi? Que propose le cours, et avec quel inconvénient?

Niveau 3. NumPy et Pandas (cours 2)

Q18. Un tableau `float64` contigu de forme `(4, 5)` : donner ses strides, ceux de sa transposée, et l'offset en octets de l'élément `[2, 3]`. Donner les strides d'un tableau `int16` contigu de forme `(3, 4, 5)`.

Q19. Donner la sortie et dire, pour `v`, `c` et `m`, s'il s'agit d'une vue ou d'une copie.

```
import numpy as np
a = np.arange(6)
v = a[1:4]; v[0] = 100
c = a[[0, 5]]; c[0] = -1
m = a[a > 3]
print(a, m, m.shape)
```

Q20. Donner la forme du résultat ou dire qu'il y a une erreur : (a) `(5, 1) + (1, 4)`; (b) `(3, 4) + (4, )`; (c) `(3, 4) + (3, )`; (d) `(2, 1, 3) + (4, 1)`; (e) `(6, ) + (1, 6)`. Pour le cas en erreur, proposer une correction.

Q21. Donner le résultat de `np.linspace(2, 4, 5)` et de `np.linspace(0, 1, 4)` avec la formule du cours. Pourquoi le cours déconseille-t-il `np.arange(0, 1.0, 0.1)`?

Q22. `X` a la forme `(200, 8)` : 200 échantillons, 8 caractéristiques. Donner la forme de `np.mean(X, axis=0, keepdims=True)`, de `np.mean(X, axis=0)` et de `np.mean(X, axis=1)`. Avec `W` de forme `(8, 3)` et `b` de forme `(1, 3)`, donner la forme de `X @ W + b` et de `np.argmax(X @ W + b, axis=1)`. Pourquoi le cours écrit-il `std + 1e-8`?

Q23. Combien de flottants 32 bits tiennent dans un registre SSE, AVX, AVX-512, NEON ? Combien de flottants 64 bits dans un registre AVX ? Décrire les étapes de $C = A + B$ en NumPy selon le cours.

Q24. Expliquer ces trois lignes du cours 2 et la raison de chacune.

```
raw["income"] = raw["income"].fillna(value=raw["income"].median())
raw["status"] = raw["status"].astype(dtype="category")
features = raw.drop(columns=["user_id"])
```

Puis, pour l'exemple `features.join(labels, how="left")` du cours (index `u1`, `u2`, `u3` ; étiquettes pour `u3` et `u1`), dire ce que reçoit `u2` et pourquoi c'est préférable à une concaténation par position.

Niveau 4. Frameworks et labs (cours 3, lab 1)

Q25. Définir SIMT et warp. Comparer CPU, GPU et TPU en une phrase chacun (optimisé pour quoi, avec quelle contrepartie).

Q26. Pourquoi appeler `cudaMalloc` à chaque tenseur temporaire ruinerait-il les performances ? Que fait l'allocateur à cache de PyTorch quand un tenseur est détruit ?

Q27. On pose $w = 2$, $x = 3$, $b = 1$, $y = 5$, $L = wx + b$ et la perte $\ell = (L - y)^2$. Calculer L , ℓ , puis $\partial\ell/\partial w$, $\partial\ell/\partial b$, $\partial\ell/\partial x$ en mode inverse. Que vaut `w.grad` après deux appels à `backward()` sur les mêmes données sans `zero_grad()` ?

Q28. Ce modèle s'exécute sans erreur mais n'apprend pas : l'optimiseur ne reçoit aucun paramètre. Pourquoi ? Proposer une correction.

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = [nn.Linear(4, 16), nn.Linear(16, 3)]
    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x
```

Q29. Relever les erreurs de cette boucle d'entraînement pour un classifieur à 3 classes.

```

model = nn.Sequential(nn.Linear(4, 3), nn.Softmax(dim=1))
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
for epoch in range(100):
    loss = criterion(model(X_train), y_train)
    loss.backward()
    optimizer.zero_grad()
    optimizer.step()
acc = (model(X_test).argmax(dim=1) == y_test).float().mean()

```

Q30. Compter les paramètres d'un MLP 784, 128, 64, 10 (trois `nn.Linear`) et d'un classifieur softmax `nn.Linear(4, 3)` pour Iris. Combien de mises à jour par époque pour 60 000 images avec `batch_size=64` ? Pour l'exemple Keras du cours (1000 échantillons, `batch_size=32`, 10 époques) ?

Q31. Comparer PyTorch et TensorFlow sur l'exécution, l'entraînement, la sérialisation et la mémoire GPU. Expliquer `define-by-run` et `define-then-run`, et ce qu'apporte `torch.compile`. Citer les trois façons de définir un modèle Keras.

Q32. Qu'est-ce qu'une fonction pure et un `pytree` en JAX ? Décrire la chaîne déclenchée par `@jax.jit`. Qu'est-ce qui rend un réseau systolique de TPU efficace ?

Q33. Un étudiant rend le lab 1 ainsi : commits sur `main`, pull request intitulée « Lab 1 solution », captures nommées `ex1.png`, `ex2.png`, pas de `neptun.txt`. Relever chaque écart avec la procédure et dire combien de points il faut pour valider ce lab.

Q34. Le générateur `SpiralDataset` du cours 2 est appelé avec `n_points_per_class=500` (3 classes). Donner la forme, les strides et le dtype de `X` et de `y`. Que fait `to_pytorch()` en mémoire ? Quel changement de type le cours 3 apporte-t-il aux étiquettes avant l'entraînement, et pourquoi ?

2. Corrigés

Niveau 1

Q1.

	Symbolique	Connexionniste
Époque	1950 à 1990	1980 à aujourd’hui, essor après 2012
Connaissance	règles et faits explicites : graphes, ontologies, si-alors	vecteurs et matrices de flottants
Raisonnement	déduction, résolution, recherche A^*	optimisation ; logique implicite dans les poids θ
Matériel	machines Lisp	GPU

Échec : impossible d’encoder à la main la variabilité infinie du réel (pas assez de if pour reconnaître un chat sous tous les éclairages et poses). C1, p. 3 et 4 ; concept 2.

Q2. (a) supervisé, $P(Y | X)$, $Y \in \{0, 1\}$; (b) non supervisé, $P(X)$ appris sur le trafic normal, alerte si $P(x_{new})$ très faible; (c) non supervisé, $P(X)$ puis échantillonnage; (d) supervisé, $P(Y | X)$; (e) non supervisé (clustering). Ce sont les exemples mêmes de C1, p. 5 ; concept 3.

Q3. Le compilateur analyse le source en AST, le compile en bytecode (mis en cache en .pyc dans __pycache__ pour les modules importés), puis la PVM, boucle infinie en C dans ceval.c, lit chaque instruction et exécute la fonction C correspondante. Pour $a + b$: type de a, type de b, recherche de __add__, appel, création d’un nouvel objet résultat. En C++ c’est une seule instruction CPU (ADD). Ce dispatch dynamique se paie à chaque instruction. C1, p. 6 ; concept 4.

Q4. Sortie obtenue : [0] [1, 2, 3, 4] [1, 2, 3, 4, 5]. b est une seconde étiquette sur la même liste : append la modifie en place. a[:] crée une copie (C1, p. 11), donc c évolue seul. a = [0] déplace l’étiquette a sans toucher l’ancienne liste, que b désigne toujours. Concept 5.

Q5. Sortie obtenue : `2 <class 'dict'> <class 'set'> False False`. `bool` est une sous-classe d'`int` (`True` vaut 1); `{}` crée un dictionnaire; `(1)` est l'entier 1 alors que `(1,)` est un tuple; la comparaison chaînée équivaut à `0 < 5 and 5 < 3`. C1, p. 9, 10 et 12; concepts 6 à 8.

Q6. $O(1)$ amorti; $O(N)$; $O(1)$; $O(N)$; $O(N)$; $O(1)$ en moyenne; $O(1)$ en moyenne. Les quatre premières valeurs et `d[k]` sont celles de C1, p. 12 et 13; les deux tests d'appartenance sont un (*rappel général*). Devoir 3 : la liste est parcourue en entier (`-1` n'y est pas), l'ensemble répond par hachage; l'écart grandit avec N . Mesure faite sur la machine de rédaction avec 1 000 000 d'éléments : environ 5 ms contre $0,1 \mu s$.

Q7.

```
python -m venv ai_lab_1
source ai_lab_1/bin/activate
pip install numpy torch
pip freeze > requirements.txt
```

L'activation modifie `$PATH` pour que `python` et `pip` soient ceux de l'environnement (installation dans son site-packages). `requirements.txt` fige les versions : `pip install -r requirements.txt` recrée un environnement identique. C1, p. 14 à 16 et 19; concept 9.

Q8. 3 classes $\times$ 100 points = 300 points et 300 appels à `math.sin` (un par point). Raisons : mémoire fragmentée (liste de listes de pointeurs, défauts de cache); surcoût de l'interpréteur à chaque appel scalaire ($N \times K$ fois); GIL qui empêche le parallélisme; listes Python impossibles à envoyer sur GPU. C1, p. 18; C2, p. 33; concept 10.

Niveau 2

Q9. Sortie obtenue : `['_A__x', '_B__x']`. Chaque `__x` est renommé avec le nom de la classe où il est écrit : deux attributs distincts coexistent. C'est le but du name mangling selon le cours : éviter les collisions de noms dans l'héritage, pas cacher. C2, p. 2; concept 11.

Q10. `__new__` reçoit `cls`, doit renvoyer une nouvelle instance, est appelé en premier; `__init__` reçoit `self`, doit renvoyer `None`. Nécessaire pour sous-classer un type immuable (`int`, `tuple`) et pour un singleton. C2, p. 5 et 6; concept 12.

Q11. MRO obtenue : `Child, Left, Right, Base, object`. Sortie : `Child, Left, Right, Base`, une par ligne. Dans `Left`, `super()` n'appelle pas `Base` mais la classe suivante de la MRO de l'objet, `Right`. `Base` n'est appelé qu'une fois. C2, p. 6 à 11; concept 13.

Q12. `SmartPhone` appelle `Camera.__init__(name, resolution)`; dans `Camera`, `super().__init__(name)` vise `Phone`, suivant dans la MRO, qui exige aussi `number`. Sortie obtenue :

```
TypeError: Phone.__init__() missing 1 required
positional argument: 'number'
```

Correction : héritage coopératif avec `**kwargs` ; chaque classe prélève ses arguments et transmet le reste par `super().__init__(**kwargs)`, l'appel initial passant tout par nom. Règle : *chaque* classe de la chaîne appelle `super()`. C2, p. 8 à 11; concepts 13 et 14.

Q13. Sortie obtenue :
1 2 1. Chaque appel à `make_counter` crée une nouvelle cellule `count` ; `c1` et `c2` ne la partagent pas. Sans `nonlocal`, `count += 1` fait de `count` une variable locale lue avant affectation : `UnboundLocalError` au premier appel (vérifié). C2, p. 11 et 12; concept 15.

Q14. Sortie obtenue :

```
decorating double
before
after
10
wrapper
```

« `decorating` » s'affiche à la définition, car `@deco` équivaut à `double = deco(double)` exécuté immédiatement. Ajouter `@functools.wraps(f)` au-dessus de `def wrapper` (avec `import functools`) pour conserver `__name__` et `__doc__`. C2, p. 12 et 15; concept 16.

Q15. {'ResNet50': ResNet50, 'VGG16': VGG16} (les classes, pas des instances). `create_model("BERT")` lève, sortie obtenue :

```
ValueError: Model BERT not found. Available:
['ResNet50', 'VGG16']
```

Avantage : chaque modèle s'enregistre lui-même, même défini dans un autre fichier ; la fabrique n'a pas à être modifiée. C2, p. 13 et 14 ; concept 16.

Q16. (a) Control ; (b) IOPub ; (c) Stdin ; (d) Heartbeat ; (e) Shell. C2, p. 16 et 17 ; concept 17.

Q17. Les VM Colab sont éphémères : tout ce qui est écrit hors d'un disque monté disparaît en fin de session. Le cours propose de monter Google Drive par FUSE ; inconvénient : chaque opération de fichier est un appel d'API réseau, avec une latence élevée par rapport à un SSD local. C2, p. 15 et 17 ; concept 17.

Niveau 3

Q18. $\text{float64} = 8 \text{ octets} : \text{strides} (5 \times 8, 8) = (40, 8) ; \text{transposée} (8, 40) ;$ offset de $[2, 3] = 2 \times 40 + 3 \times 8 = 104 \text{ octets}.$ $\text{int16} (2 \text{ octets}), \text{forme} (3, 4, 5) : (4 \times 5 \times 2, 5 \times 2, 2) = (40, 10, 2).$ Strides vérifiés avec NumPy. C2, p. 18 et 19 ; concept 18.

Q19. Sortie obtenue : `[ 0 100 2 3 4 5] [100 4 5] (3,).` `v` est une vue (tranche) : écrire 100 dans `v[0]` modifie `a[1]`. `c` est une copie (liste d'indices) : `a[0]` reste 0. `m` est une copie 1D par masque, calculée après la modification, d'où le 100. C2, p. 19 et 20 ; concept 19.

Q20. (a) (5,4) ; (b) (3,4) ; (c) erreur : (3,) devient (1,3), et $4 \neq 3$; (d) (2,4,3) ; (e) (1,6). Correction de (c) : `reshape(3, 1)`, qui donne (3,4). Toutes les formes vérifiées avec NumPy. C2, p. 21 à 23 ; concept 21.

Q21. $x_i = 2 + i \cdot (4 - 2) / 4 : [2. \ 2.5 \ 3. \ 3.5 \ 4.].$ $x_i = i / 3 : [0. \ 0.33333333 \ 0.66666667 \ 1.]$ (sorties obtenues). `arange` additionne un pas ; 0,1 n'étant pas représentable exactement en binaire, l'erreur s'accumule et la borne peut dériver. `linspace` divise l'intervalle et garantit la borne finale. C2, p. 24 et 25 ; concept 22.

Q22. (1,8); (8,); (200,); $X @ W + b$: (200,3) (biais diffusé sur les 200 lignes); `argmax(axis=1)` : (200,), une classe par échantillon (formes vérifiées). 10^{-8} évite une division par zéro si une caractéristique est constante ($\sigma = 0$). C2, p. 26 et 27 ; concept 22.

Q23. SSE 4, AVX 8, AVX-512 16, NEON 4 (C2, p. 21). En float64, AVX : $256/64 = 4$ (*rappel général*). $C = A + B$: Python ne boucle pas, délègue à une fonction C, qui détecte les capacités du CPU, charge avec AVX2 8 flottants de A et 8 de B dans des registres YMM, exécute `vaddps` et écrit le résultat. Concept 20.

Q24. `fillna` impute les valeurs manquantes de `income` par la médiane : la plupart des modèles refusent les NaN, et on garde l'échantillon. `astype("category")` remplace les chaînes par des codes entiers et un dictionnaire des valeurs : mémoire réduite de 90 à 95 % pour une faible cardinalité. `drop(columns=["user_id"])` retire un identifiant non prédictif, source de fuite de données et de mémorisation. Jointure : d'après le support, `u2` reçoit NaN ; par position, `u1` aurait reçu l'étiquette de `u3` sans aucune erreur. C2, p. 29 à 31 ; concepts 23 et 24.

Niveau 4

Q25. SIMT : une instruction diffusée à de nombreux threads qui l'exécutent en même temps sur des données différentes. Warp : groupe de threads (typiquement 32) qui exécutent la même instruction. CPU : optimisé pour la latence et le contrôle de flux complexe, peu de grosses ALU. GPU : optimisé pour le débit, flexible, mais relit et réécrit ses résultats intermédiaires en mémoire. TPU : ASIC à réseau systolique, meilleur débit et rendement par watt sur les produits matriciels denses, sans contrôle de flux dynamique. C3, p. 1, 2, 14 et 15 ; concepts 26 et 34.

Q26. `cudaMalloc` passe en espace noyau et impose une synchronisation globale CPU et GPU : le pipeline GPU se vide à chaque appel. PyTorch réserve de grands blocs au démarrage ; un tenseur détruit rend sa mémoire au pool interne, pas à CUDA, et le bloc est réattribué au noyau suivant du même stream sans attendre. C3, p. 2 et 3 ; concept 27.

Q27. $L = 2 \times 3 + 1 = 7$; $\ell = (7 - 5)^2 = 4$. Gradient amont $\partial \ell / \partial L = 2(L - y) = 4$, multiplié par les dérivées locales : $\partial \ell / \partial w = 4 \times x = 12$, $\partial \ell / \partial b = 4 \times 1 = 4$, $\partial \ell / \partial x = 4 \times w = 8$. Sans `zero_grad()`, `backward()` accumule : `w.grad` vaut $12 + 12 = 24$. Calcul (*rappel général*) appliqué au mécanisme de C3, p. 4 et 7; concept 28.

Q28. Une liste Python ordinaire n'est pas interceptée par `__setattr__` de `nn.Module` : les couches n'entrent pas dans `_modules`, donc `model.parameters()` ne les voit pas (C3, p. 5; concept 29). Correction : affecter chaque couche à un attribut (`self.fc1 = nn.Linear(4, 16)`), ou utiliser `nn.Sequential` comme en C3, p. 16. `nn.ModuleList` répond aussi au problème (*rappel général*).

Q29.

- `nn.Softmax` avant `CrossEntropyLoss`, qui attend des logits bruts (C3, p. 17) : retirer la couche softmax.
- `zero_grad()` après `backward()` efface les gradients juste calculés : `step()` ne met rien à jour. Ordre correct : `zero_grad`, `backward`, `step`.
- Évaluation sans `model.eval()` ni `torch.no_grad()` : le DAG est construit inutilement et un éventuel dropout resterait actif.

Concept 30.

Q30. $784 \times 128 + 128 = 100\,480$; $128 \times 64 + 64 = 8\,256$; $64 \times 10 + 10 = 650$; total 109 386. `nn.Linear(4, 3) :  $4 \times 3 + 3 = 15$` . Mises à jour : $\lceil 60\,000 / 64 \rceil = 938$ par époque; Keras : $\lceil 1000 / 32 \rceil = 32$ par époque, 320 en tout. Formules (*rappel général*) ; 60 000 est la taille usuelle du jeu d'entraînement MNIST (*rappel général*). Concepts 29 et 32.

Q31.

- Exécution : PyTorch dynamique (define-by-run, le graphe se construit en exécutant le Python); TensorFlow eager par défaut en TF2, graphe statique via `@tf.function` (define-then-run : on assemble le graphe, puis on l'exécute).
- Entraînement : boucle explicite en PyTorch; `model.fit()` en Keras.
- Sérialisation : `state_dict` pickle en `.pt` (Python et la classe requies); `SavedModel` en Protocol Buffers.

- Mémoire GPU : allocateur à cache sobre ; TensorFlow prend toute la VRAM.
- `torch.compile` (PyTorch 2) : TorchDynamo intercepte l'exécution et fusionne les opérations en noyaux C++ ou Triton, sans surcoût Python.
- Keras : Sequential, API fonctionnelle, sous-classement de `keras.Model`.

C3, p. 7 à 13 ; concepts 31 à 33.

Q32. Fonction pure : sortie dépendant uniquement des entrées, sans effet de bord ; les poids ne sont jamais modifiés en place. Pytree : imbrication de dictionnaires et listes dont les feuilles sont des tableaux ; JAX sépare la structure des données et ne passe au compilateur que les tableaux à plat. `@jax.jit` : traçage en Jaxpr, abaissement en StableHLO, compilation et fusion par XLA en code machine (PTX, code TPU). Réseau systolique : poids fixés dans la grille, entrées qui circulent, résultats intermédiaires passés d'ALU en ALU sans mémoire globale. C3, p. 13 à 15 ; concept 34.

Q33.

- Travail sur `main` : il faut une branche `solution`.
- PR intitulée `lab1`, source `solution`, cible `main`, assignée à `balazskvancz`, laissée ouverte.
- Captures aux noms exacts `f1_1.png`, `f1_2.png`, `f1_3.png`, `f2_1.png`, `f2_2.png`, `f3.png`, `f4.png`, sinon rendu refusé.
- `neptun.txt` absent : le contrôle GitHub Actions échoue.

Validation : 8 points sur 20. Un rendu hors procédure n'est pas évalué. L1 ; concept 36.

Q34. `x` : forme (1500,2), `float32`, `strides` (8,4) ; `y` : forme (1500,), `uint8`, `stride` (1,) (vérifié avec NumPy). `to_pytorch()` appelle `torch.from_numpy`, qui crée des tenseurs partageant le tampon NumPy, sans copie (C2, p. 35 et 37). Le cours 3 génère `y` en `np.int64` (C3, p. 16) : `CrossEntropyLoss` attend des indices de classe entiers longs (*rappel général*). Concepts 25 et 35.